PRACTICAL: Feature Branch Creation, Merging and Conflict Resolution

Step 1: Clone the Existing Repository

```
git clone https://github.com/<username>/data-science-project.git
cd data-science-project
```

Step 2: Check Existing Branches `git`

```
branch
```

Output:

```
* main
```

Step 3: Create a Feature Branch

Create a branch for developing a new feature.

```
git checkout -b feature-data-analysis
```

Output:

```
Switched to a new branch 'feature-data-analysis'
```

Step 4: Create and Modify Project Files

Create a file for the new feature.

```
echo "Data Analysis Feature" > analysis.py
```

Add and commit the changes.

```
git add .
git commit -m "Added data analysis feature"
```

Output:

```
[feature-data-analysis abc1234] Added data analysis feature
1 file changed create mode
100644 analysis.py
```

Step 5: Create Another Feature Branch

Switch back to the main branch.

```
git checkout main
```

Create another feature branch. `git`

```
checkout -b feature-visualization
```

Output:

```
Switched to a new branch 'feature-visualization'
```

Step 6: Add Changes to the Second Feature Branch Create a visualization file.

```
echo "Data Visualization Feature" > visualization.py
```

Add and commit the changes.

```
git add .
git commit -m "Added data visualization feature"
```

Output:

```
 [feature-visualization def5678] Added data visualization feature
1 file changed create mode 100644
visualization.py
```

Step 7: View All Branches

```
git branch
```

Output:

```
 feature-data-analysis
* feature-visualization
main
```

This shows that different features are being developed in separate branches.

Step 8: Merge the First Feature Branch

Switch to the main branch.

```
git checkout main
```

Merge the data analysis branch. `git`

```
merge feature-data-analysis
```

Output:

```
Updating abc1234..def5678
Fast-forward
analysis.py | 1 +
1 file changed
```

Step 9: Merge the Second Feature Branch `git`

```
merge feature-visualization
```

Output:

```
Merge made by the 'ort' strategy.
visualization.py | 1 + 1 file
changed
```

Step 10: Create a Merge Conflict

Modify the same line of a file in two different branches.

For example, change the same line in `README.md` in both branches.

```
<<<<<<< HEAD
Data Science Project - Main Branch
=======
Data Science Project - Feature Branch
>>>>>>> feature-data-analysis
```

The conflict occurs because Git cannot automatically decide which change should be kept.

Step 11: Resolve the Conflict

Open the conflicted file and manually select the required content.

Remove the conflict markers:

```
<<<<<<< HEAD
=======
>>>>>>> feature-data-analysis
```

Save the corrected file.

Step 12: Add the Resolved File `git`

```
add README.md
```

Step 13: Commit the Conflict Resolution `git`

```
commit -m "Resolved merge conflict"
```

Output:

```
[main abc7890] Resolved merge conflict
```

Step 14: Push the Merged Project to GitHub

```
git push origin main
```

Output:

```
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Writing objects: 100% (10/10), done.
To https://github.com/<username>/data-science-project.git
 abc1234..abc7890 main -> main
```

Step 15: Verify the Branches on GitHub

Open the GitHub repository and check the **Branches** section.

Output:

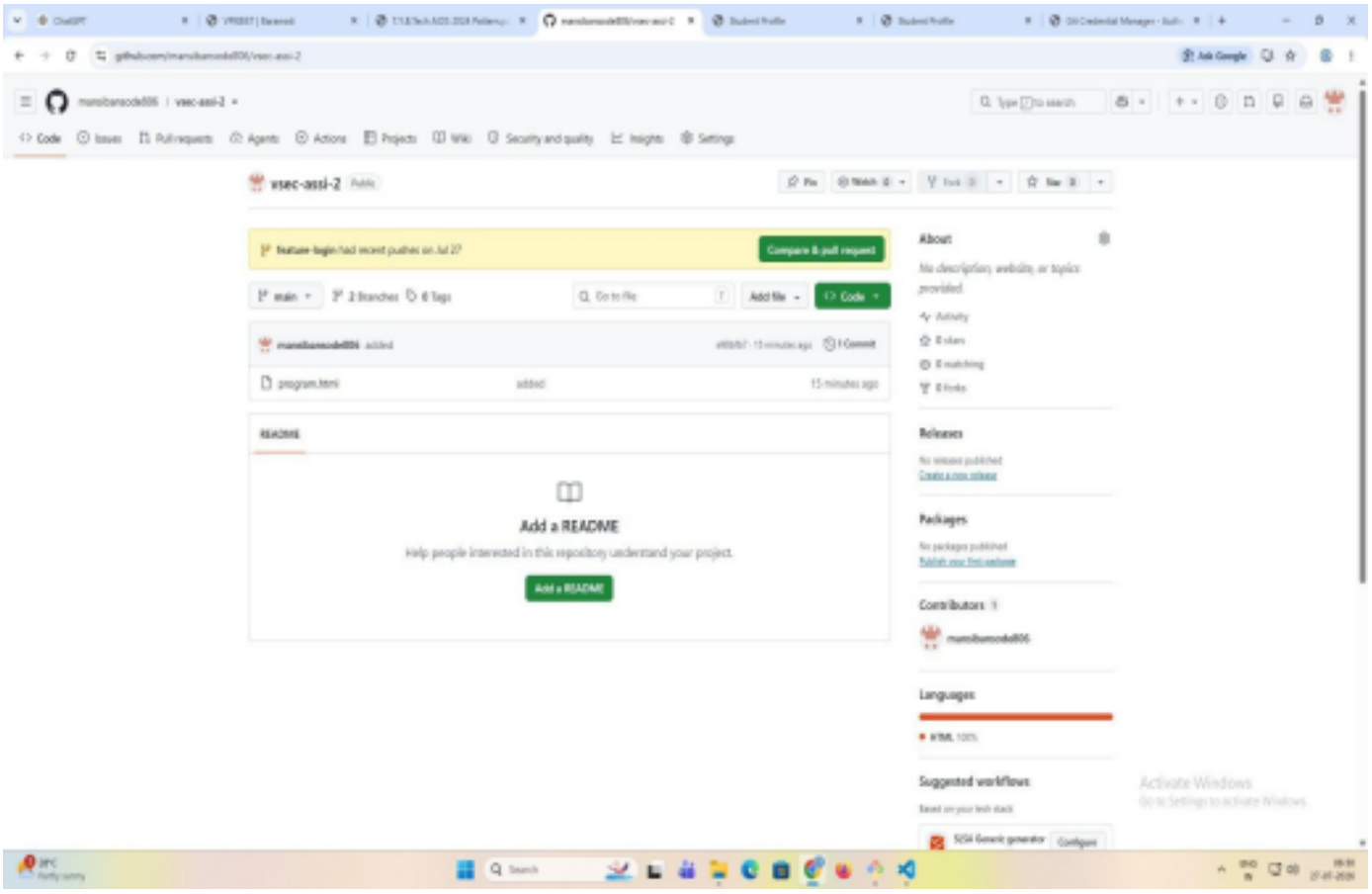
```
main
feature-data-analysis feature
visualization
```

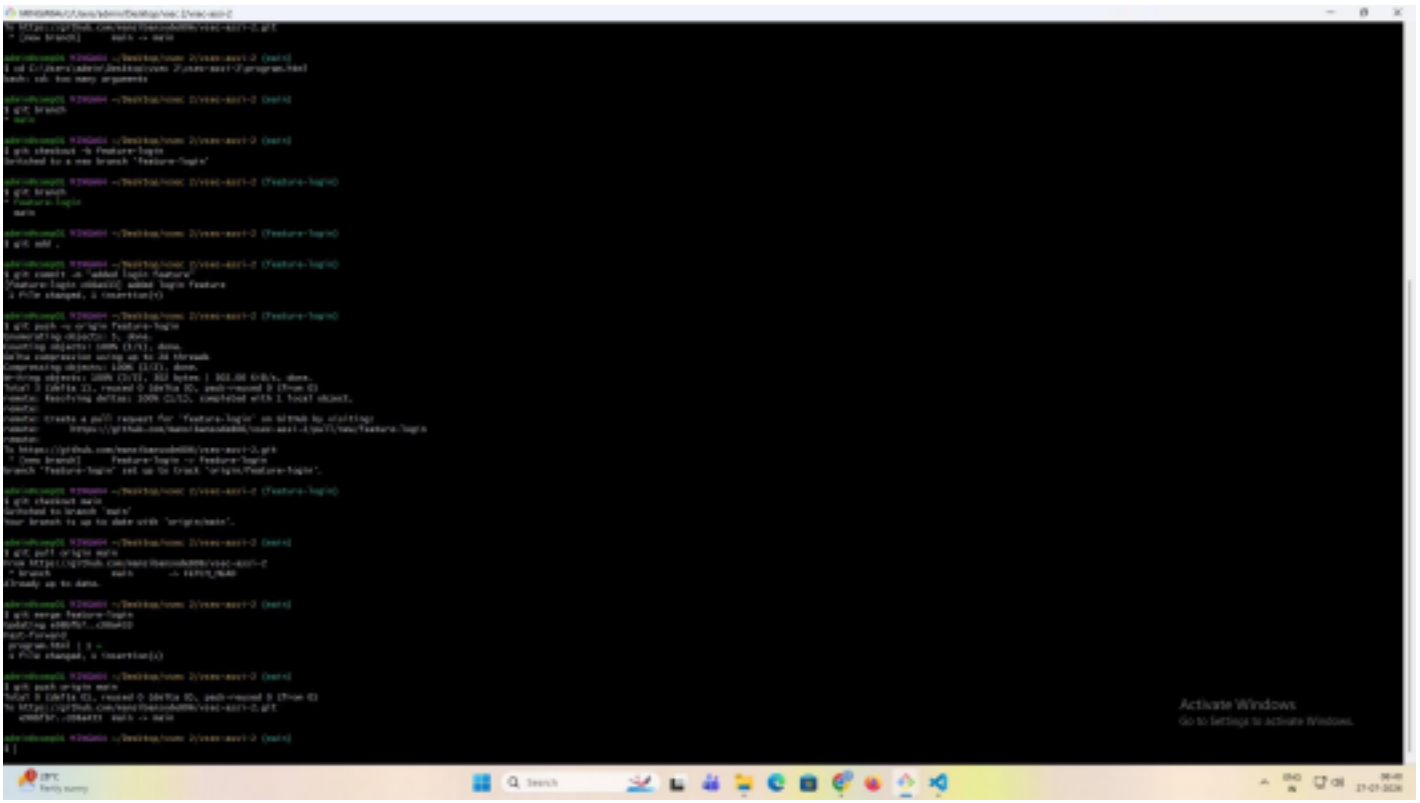
The feature branches can be used as separate workspaces, while the completed

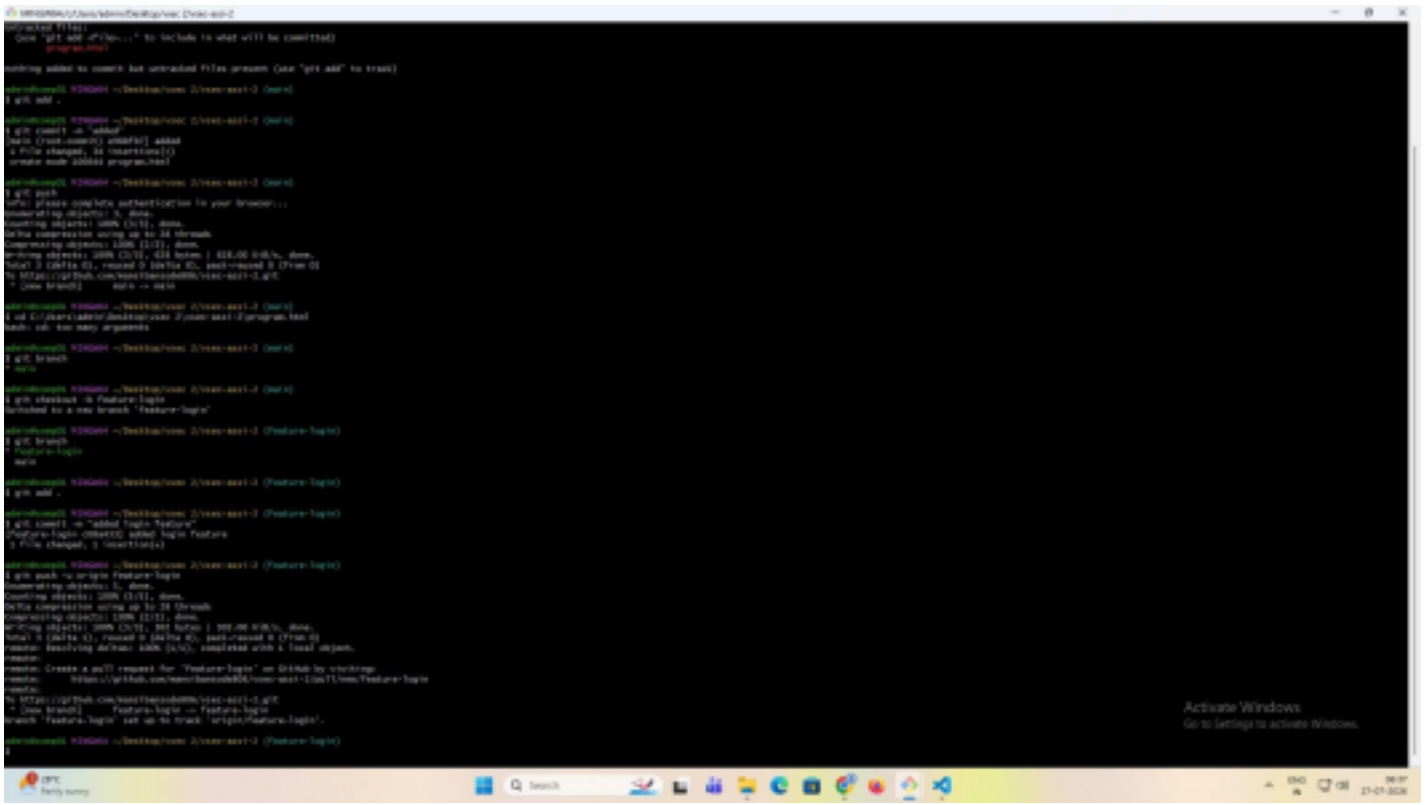
features can be merged into the main branch.

Result

The feature branches were successfully created and used for separate development work. The branches were merged into the main branch, and a merge conflict was successfully identified and resolved.






Activate Windows
Go to Settings to activate Windows.